

10Base-T1S Converter 보드

사용자 매뉴얼

Rev 0.2

티에스엔랩(주)



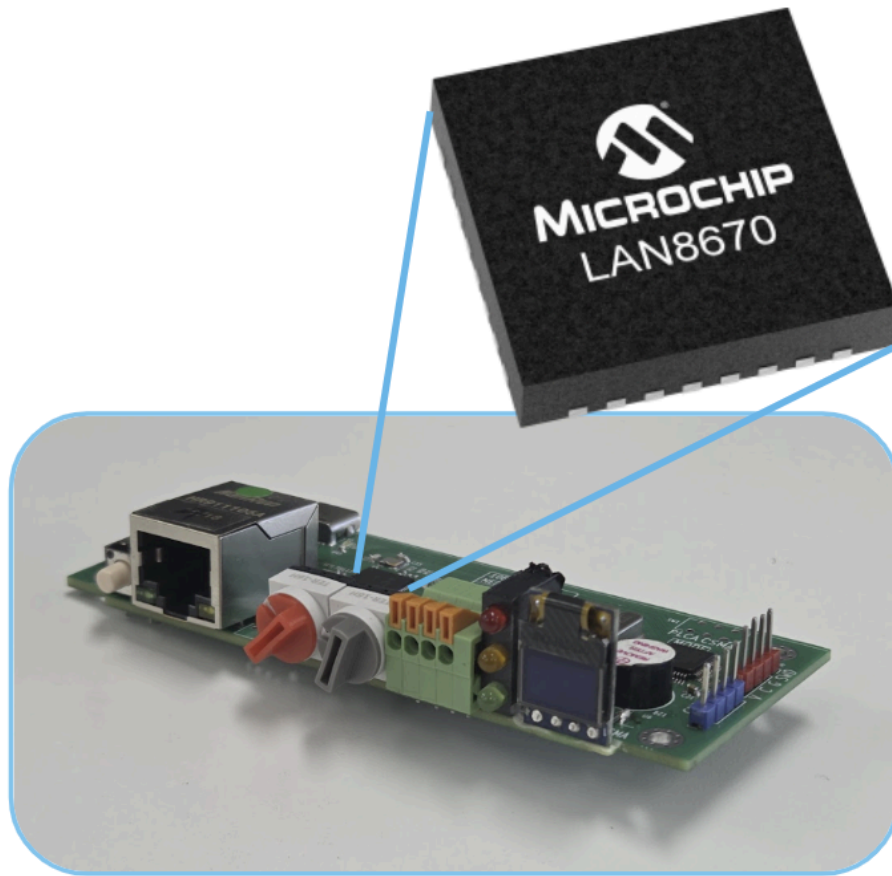
Real Tech for Real Time

(이 페이지는 의도적으로 비워 놓았습니다.)

1. 개요.....	3
2. 인터페이스.....	5
3. 사용 방법.....	7
3.1. Mode Configuration.....	7
3.2. DIP Switch.....	8
3.3. Error Code.....	9
4. 사용 예제.....	11
4.1. 일반 PC를 활용한 10Base-T1S 네트워크 통신.....	12
4.2. 일반 PC를 활용한 10Base-T1S 네트워크 패킷 스니핑.....	18
4. 이슈 리스트.....	25
4.1. ETH 포트 전원 이슈.....	25
4.2. 스니퍼 & 코디네이터 모드 충돌 이슈.....	25
4.3. 양방향 통신 대역폭 이슈.....	25

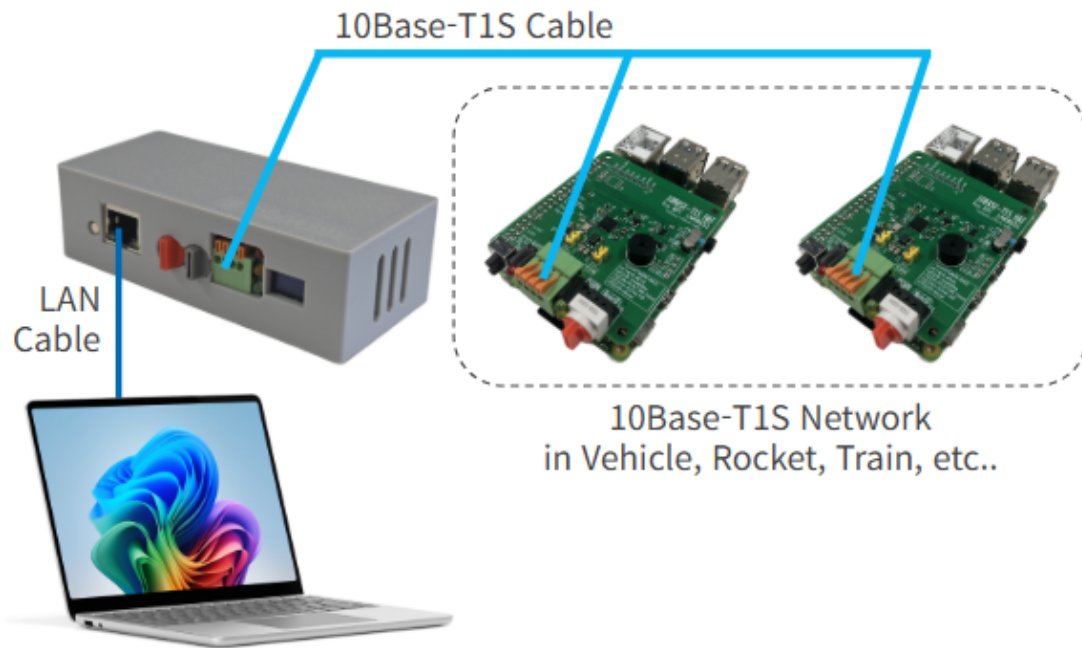
(이 페이지는 의도적으로 비워 놓았습니다.)

1. 개요



<10Base-T1S Converter 보드>

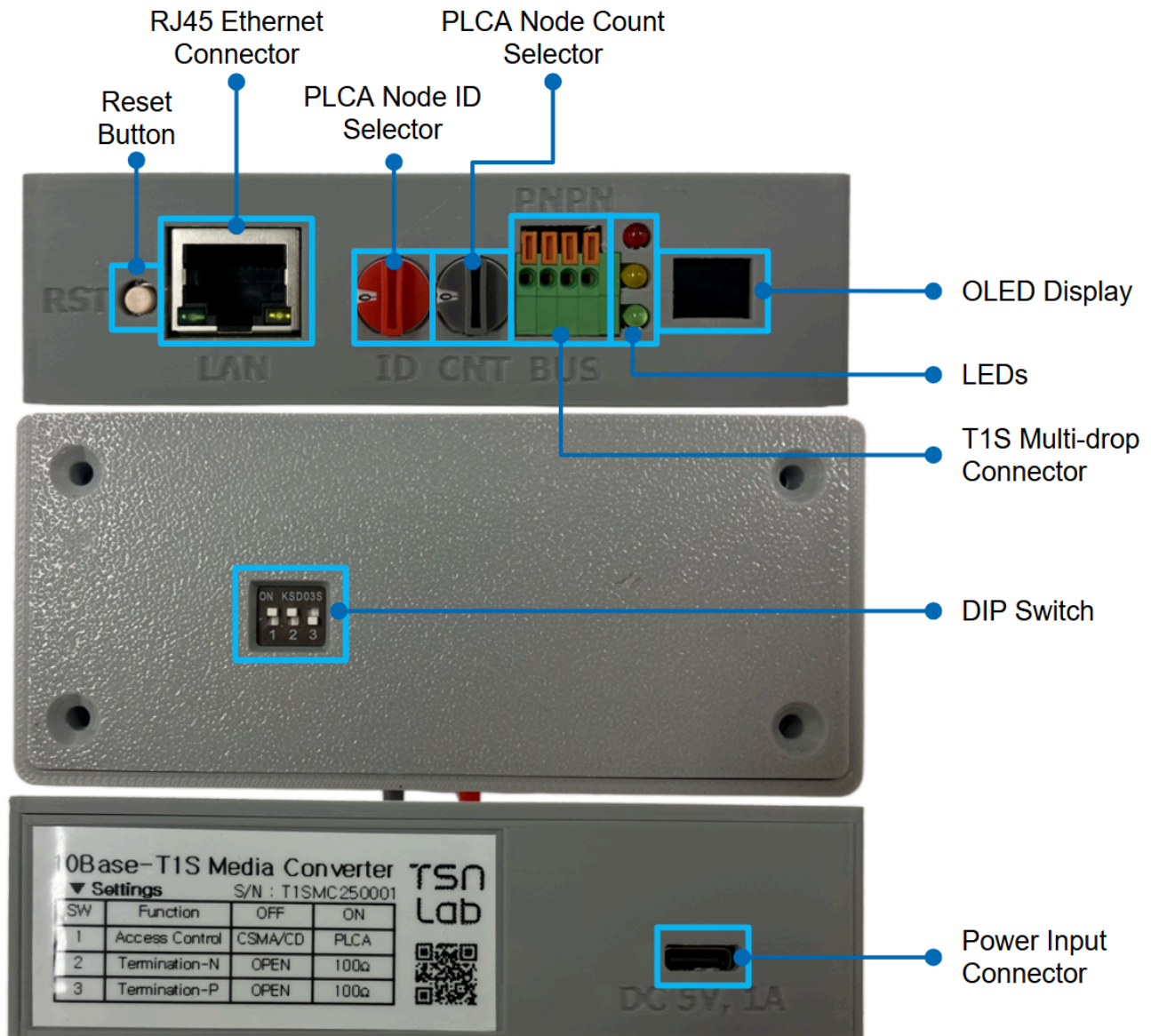
- 10Base-T1S Converter(이하, Converter) 보드는 단일쌍 케이블을 통해 10Mbps의 반이중 이더넷 기술로 전송할 수 있는 LAN8670 PHY칩 기반의 10Base-T1S LAN 세그먼트와 4-pair 케이블을 사용하는 범용 고속 이더넷 망과의 연동을 지원하는 컨버터입니다.
- USB 포트를 통한 간편한 전원 공급 뿐만 아니라, PLCA 동작을 위한 Node ID와 Node Count 설정용 회전식 스위치, 사용자 편의를 위한 LED 인디케이터, 그리고 OLED가 전면에 장착되어 있습니다.
- 10Base-T1S Converter 보드에서 지원하는 주요 기능은 다음과 같습니다.
 - 802.3cg - 2019
 - 10Mbps T1S Link Speed
 - PLCA 기반 Half-duplex 이더넷 동작모드
 - 최대 8개의 PHY를 포함한 Half-duplex Multi-Drop을 최소 25m까지 지원



<10Base-T1S Converter의 활용 목적 예시>

일반적인 PC를 Converter 보드와 LAN Cable로 연결할 수 있으며, 이를 활용해 PC에서는 10Base-T1S 네트워크 상에 흐르는 이상 데이터를 확인 할 수 있습니다. 또한 PC에서 10Base-T1S 네트워크로 데이터를 흘려보내 프로토콜 시험도 가능합니다(= 10Base-T1S 게이트웨이 역할 수행 가능).

2. 인터페이스



<10Base-T1S Converter 보드 인터페이스>

10Base-T1S Converter 보드는 T1S 통신을 지원하기 위해 다음과 같은 여러 인터페이스를 제공합니다.

- Reset Button
 - 보드를 재시작 할 수 있도록 제공되는 외부 버튼
- RJ45 Ethernet Connector
 - 100Base-TX Ethernet 연결 단자
- PLCA Node ID Selector
 - T1S 버스의 노드를 ID를 선택하기 위한 회전식 스위치

- 총 16개의 ID(0x0~0xF)를 지원
- PLCA Node Count Selector
 - T1S 버스 상의 노드의 총 개수를 지정하기 위한 회전식 스위치
 - 총 16개의 Count(0x0~0xF)를 지원
- OLED Display
 - 동작 상태 표시용 OLED Display
- LEDs
 - AWAKE, Link Activity, PLCA 활성 상태 출력 LED
- T1S Multi-drop Connector
 - T1S 버스 중 이웃 노드와의 간편한 연결을 위한 4-Pin Connector
- DIP Switch
 - T1S 노드의 충돌/회피 알고리즘 설정
 - DIP Switch의 1번
 - PLCA와 CSMA 알고리즘 선택 가능
 - 10Base-T1S의 표준은 PLCA임으로 PLCA를 사용하는 것을 추천
 - T1S 버스의 종단 저항 설정
 - DIP Switch의 2, 3번
- Power Input Connector
 - USB C Type 전원 입력 단자
 - DC 5V, 1A 입력

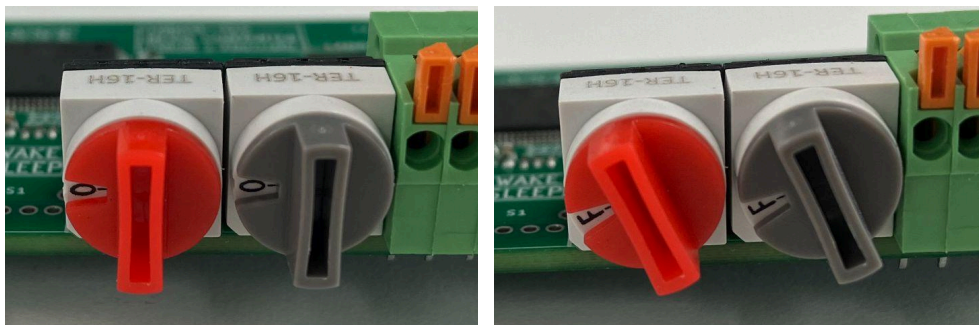
3. 사용 방법

3.1. Mode Configuration

Converter 보드는 10Base-T1S 통신을 위한 다음과 같은 여러 모드를 지원합니다.

- 코디네이터(Coordinator)
 - 10Base-T1S 버스의 통신에 참여하여 PLCA 마스터의 역할을 하는 모드
※ PLCA 마스터는 버스 상에 오직 하나만 존재
- 팔로워(Follower)
 - 10Base-T1S 버스의 통신에 참여하여 PLCA 슬레이브의 역할을 하는 모드
- 스니퍼(Sniffer)
 - 10Base-T1S 버스의 통신에 참여하지 않으며, 오로지 패킷 스니핑만 하는 모드

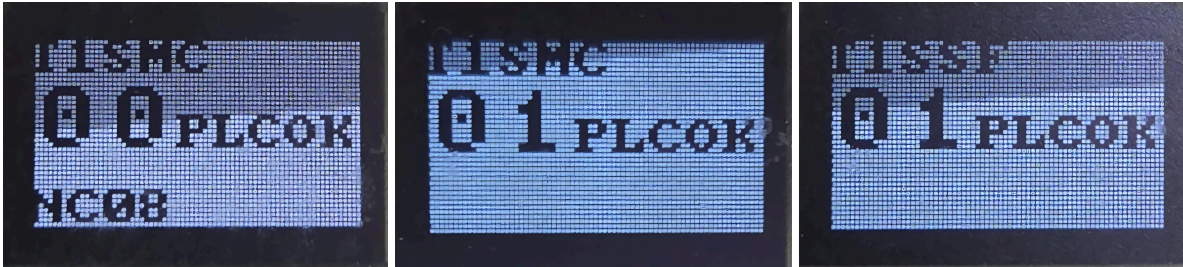
이러한 각 모드는 Converter 보드의 Node ID와 Node Counter 설정을 위한 회전식 스위치를 통해 변경할 수 있습니다.



<Node ID와 Node Count 설정을 위한 10Base-T1S Converter 보드의 회전식 스위치>

- 코디네이터 모드 설정 방법
 - Node ID를 0으로 설정
 - Node Count를 1 이상의 값으로 설정
 - PLCA는 Node ID 0을 코디네이터로 인식함
- 팔로워 모드 설정 방법
 - Node ID를 1 이상의 값으로 설정
 - Node Count를 1 이상의 값으로 설정
 - PLCA는 Node ID 1 이상을 팔로워로 인식함
- 스니퍼 모드 설정 방법

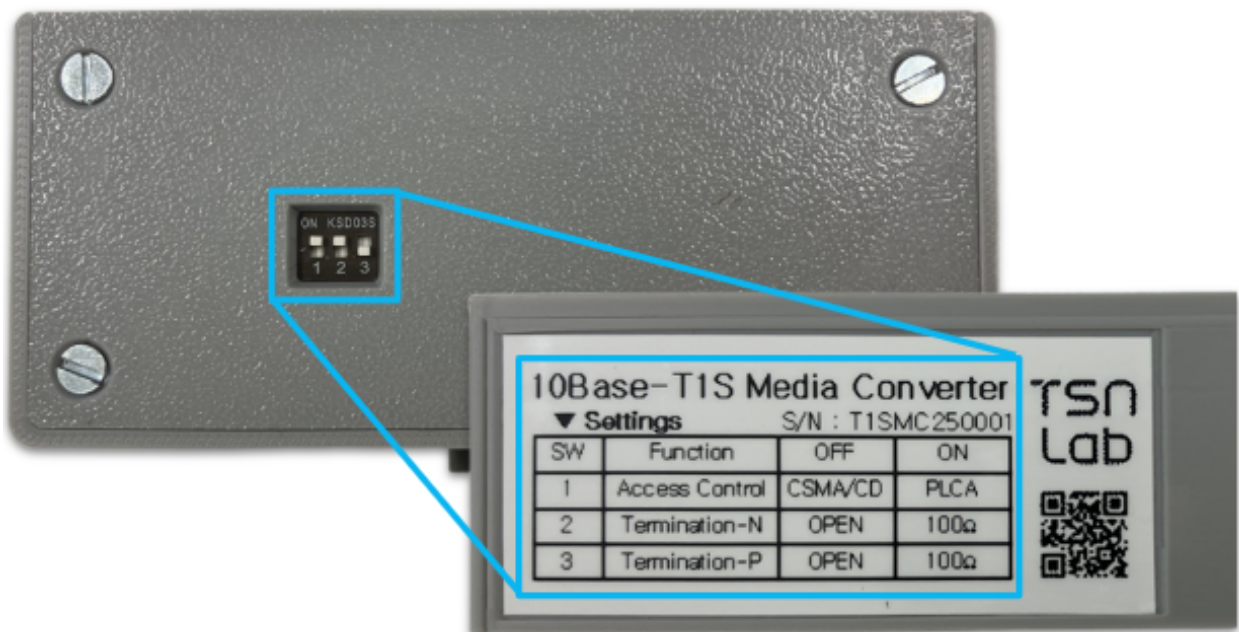
- Node ID를 1 이상의 값으로 설정
- Node Count를 0으로 설정
- 스니퍼 모드는 오로지 흘러가는 패킷을 스니핑만 수행함



<왼쪽부터 차례대로 Coordinator, Follower, Sniffer 모드 OLED 출력>

3.2. DIP Switch

Converter 보드는 버스 통신 상에 발생될 충돌을 방지하기 위해 두가지 충돌/회피 알고리즘(PLCA, CSMA)과 버스 상의 종단 노드를 표시하기 위한 종단 저항(이하, TERM)을 제공합니다. 이는 Converter 보드의 DIP Switch를 통해서 설정할 수 있습니다.

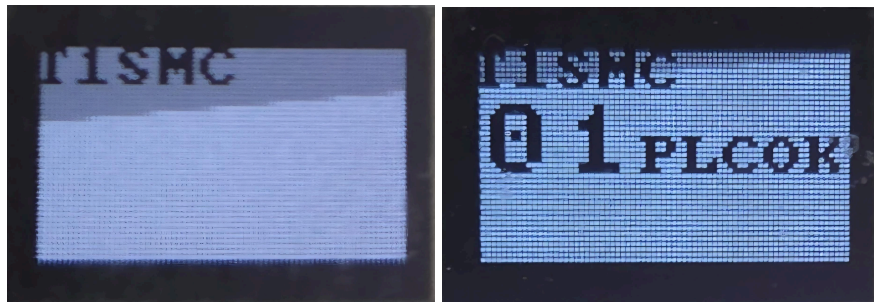


<10Base-T1S Converter 보드의 DIP Switch>

- DIP Switch 1
 - ON: PLCA 충돌/회피 알고리즘 사용
 - OFF: CSMA 충돌/회피 알고리즘 사용

※ DIP Switch 1은 Default로 ON으로 두어서 PLCA를 사용하는 것을 권장합니다.

- DIP Switch 2
 - ON: MDI Negative 100Ω TERM
 - OFF: MDI Negative Open
- DIP Switch 3
 - ON: MDI Positive 100Ω TERM
 - OFF: MDI Positive Open



<(좌) CSMA가 설정되었을 때 OLED 출력 / (우) PLCA로 설정되었을 때 OLED 출력>

DIP Switch 1을 OFF로 두었을 때에는 그림의 좌측과 같이 OLED에 출력되며, ON으로 두었을 때에는 그림의 우측과 같이 OLED에 출력됩니다.

3.3. Error Code

Converter 보드는 통신 상의 이상이 발생하였을 때 OLED에 Error Code가 함께 출력되며, 각 Error Code에대한 의미는 다음과 같습니다.

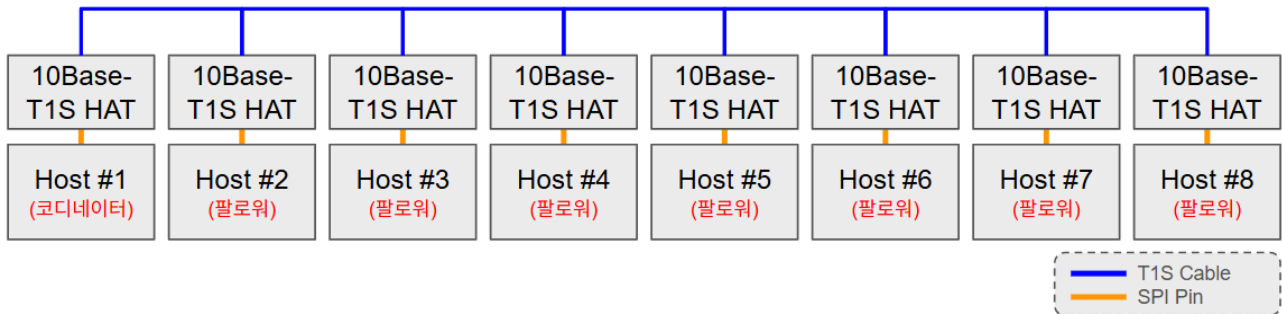


<OLED에 출력되는 Error Code 예시>

- ERR = 0, 1
 - No Error
- ERR = 2
 - 중복된 Node ID를 가진 노드가 버스 상에 존재함
 - PLCA 설정에서 출력되는 에러
- ERR = 3

- 코디네이터(Node ID = 0)가 PLCA 시작을 위한 비콘을 송신하지 않았는데 비콘을 수신받을 경우 발생하는 에러
- PLCA 설정 중 코디네이터 모드에서만 출력되는 에러
- ERR = 4
 - Node Count 값이 버스 상의 최대 Node ID 보다 작게 설정된 경우 발생하는 에러
 - PLCA 설정 중 코디네이터 모드에서만 출력되는 에러
- ERR = 5
 - 버스 상에 코디네이터가 존재하지 않을 경우 발생하는 에러
 - PLCA 설정에서 출력되는 에러
- ERR = 6
 - 버스 상에 코디네이터가 두 개 이상 존재하는 경우 발생하는 에러
 - PLCA 설정에서 출력되는 에러

4. 사용 예제



<단일 차동 페어로 연결된 하나의 10Base-T1S 네트워크 예시>

10Base-T1S는 단일 차동 페어(Single Differential Pair)를 사용하는 멀티드롭 이더넷 물리 계층으로, 하나의 네트워크에 여러 노드가 버스 구조로 연결될 수 있습니다. 이 구조에서 네트워크의 시간 기준과 동작 안정성을 확보하기 위해 코디네이터와 팔로워를 활용합니다. 하나의 10Base-T1S 네트워크에는 오로지 하나의 코디네이터만 존재하며 그 외에는 모두 팔로워 역할을 수행합니다.

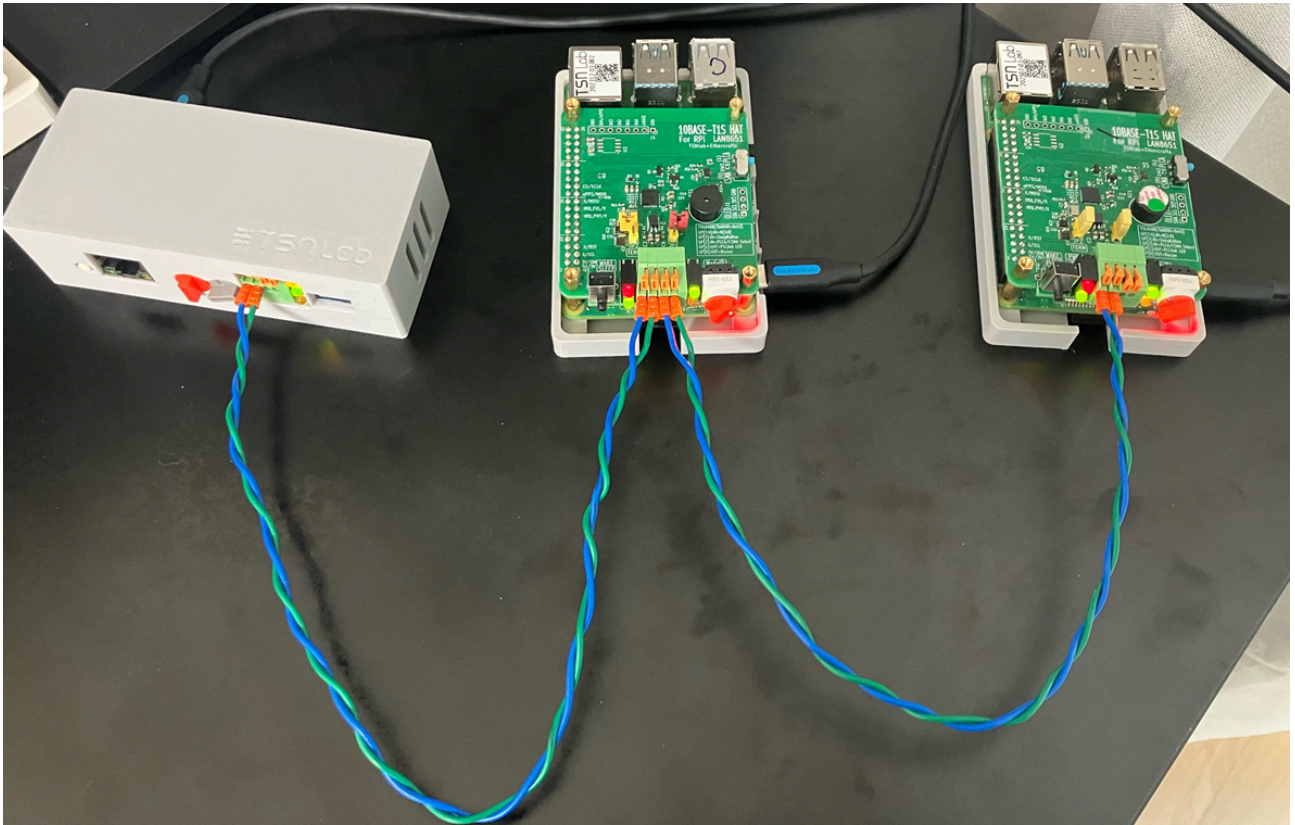
- 코디네이터(Coordinator)
 - 10Base-T1S 네트워크에서 논리적인 기준 노드로 동작
 - 네트워크 시간 기준을 제공하며 PLCA 동작의 기준을 설정
 - 모든 팔로워가 동기화할 수 있는 기준 신호인 Beacon을 제공
- 팔로워(Follower)
 - 코디네이터의 타이밍과 제어 신호에 동기화되어 동작
 - 본인이 할당된 PLCA 슬롯에서만 프레임 송신
 - 할당된 PLCA 슬롯에서만 송신하므로 네트워크 충돌 방지
- 스니퍼(Sniffer)

※ 해당 모드는 오직 본 10Base-T1S Converter 상품에서만 지원하는 모드입니다.

- 스니퍼는 팔로워와 동일한 역할을 수행 (코디네이터에 동기화)
- 팔로워는 송수신이 가능하지만 스니퍼는 오로지 수신만 가능

본 예제에서는 10Base-T1S Converter 보드의 코디네이터 혹은 팔로워 모드를 활용해 일반 PC(e.g. 노트북 등)와 10Base-T1S 노드(e.g. 10Base-T1S HAT 보드)간에 IP 혹은 TCP, UDP 통신을 수행하는 과정과 통신 시 발생하는 패킷을 스니핑하는 방법을 설명합니다.

4.1. 일반 PC를 활용한 10Base-T1S 네트워크 통신

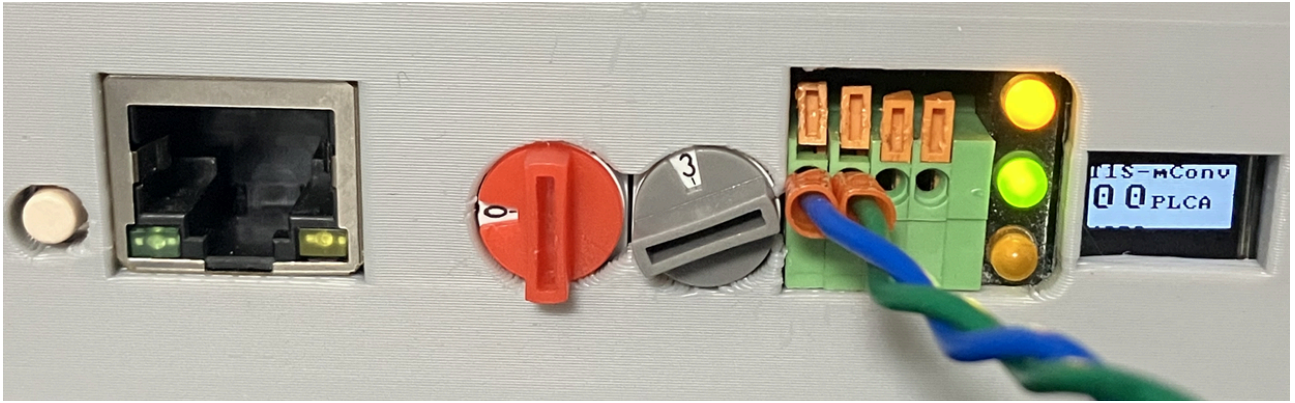


<버스 구조의 10Base-T1S 네트워크 통신 예제>

본 4.1. 과정에서는 일반 PC가 Converter 보드를 활용하여 10Base-T1S 네트워크에 참여해 네트워크 내에 존재하는 노드와 통신을 수행하는 예제를 설명합니다. 이를 위한 10Base-T1S 네트워크를 형성하기 위해서 Converter 보드(= 10Base-T1S Converter) 한 개와 HAT 보드(= 10Base-T1S HAT) 두 개를 사용합니다.

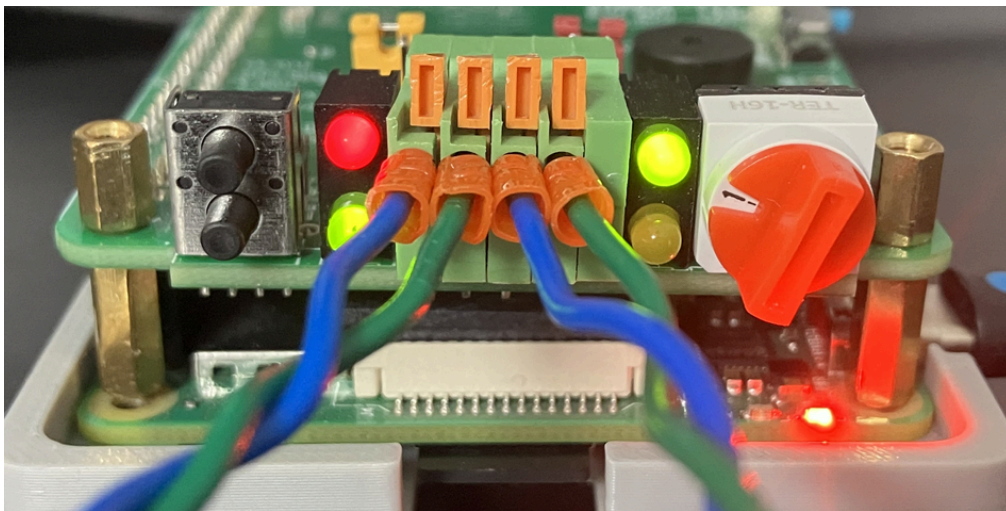
※ 일반 PC로는 MSI의 Modern 14 모델의 노트북을 사용하였으며, 노트북의 운영체제로는 Linux/Ubuntu 24.04 LTS를 활용하였습니다.

※ 유의사항: 10Base-T1S 네트워크 환경 구성은 사용자 환경에 맞춰 구성해주시면 됩니다. 만약, 10Base-T1S HAT 보드를 사용하고 계신다면 [HAT 보드 사용자 매뉴얼](#)을 참고해주시기 바랍니다.



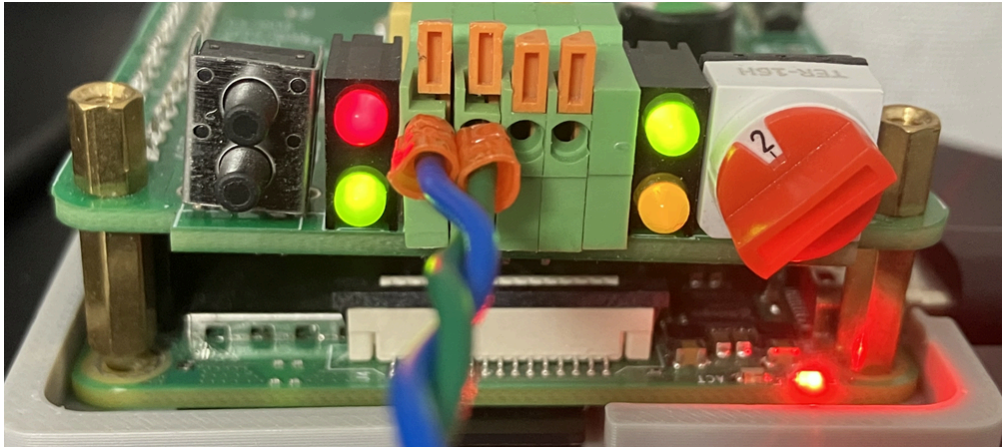
<10Base-T1S 네트워크 중 가장 왼쪽에 위치한 Converter 보드의 인터페이스>

Converter 보드의 Node ID는 0으로 설정하였으며, Node Count는 3으로 설정하였습니다. 이 값이 의미하는 것은 “Converter 보드가 10Base-T1S 네트워크의 코디네이터 역할을 수행하며 이 네트워크에는 총 3개의 노드(코디네이터 포함)가 존재한다.”와 같습니다. 또한, Converter 보드의 RJ-45 규격의 LAN 인터페이스는 아직 일반 PC와 연결하지 않은 상태입니다.



<10Base-T1S 네트워크 중 중간에 위치한 HAT 보드의 인터페이스>

중간에 위치한 HAT 보드의 Node ID는 1로 설정하였습니다(HAT의 경우 Node Count는 8로 고정). 이 값이 의미하는 것은 “HAT 보드가 10Base-T1S 네트워크의 팔로워 역할을 수행하며 코디네이터의 신호에 맞춰 통신을 수행한다.”와 같습니다.



<10Base-T1S 네트워크 중 가장 오른쪽에 위치한 HAT 보드의 인터페이스>

오른쪽 끝에 위치한 HAT 보드의 Node ID는 2로 설정하였습니다(HAT의 경우 Node Count는 8로 고정). 이 값이 의미하는 것은 “HAT 보드가 10Base-T1S 네트워크의 팔로워 역할을 수행하며 코디네이터의 신호에 맞춰 통신을 수행한다.”와 같습니다.

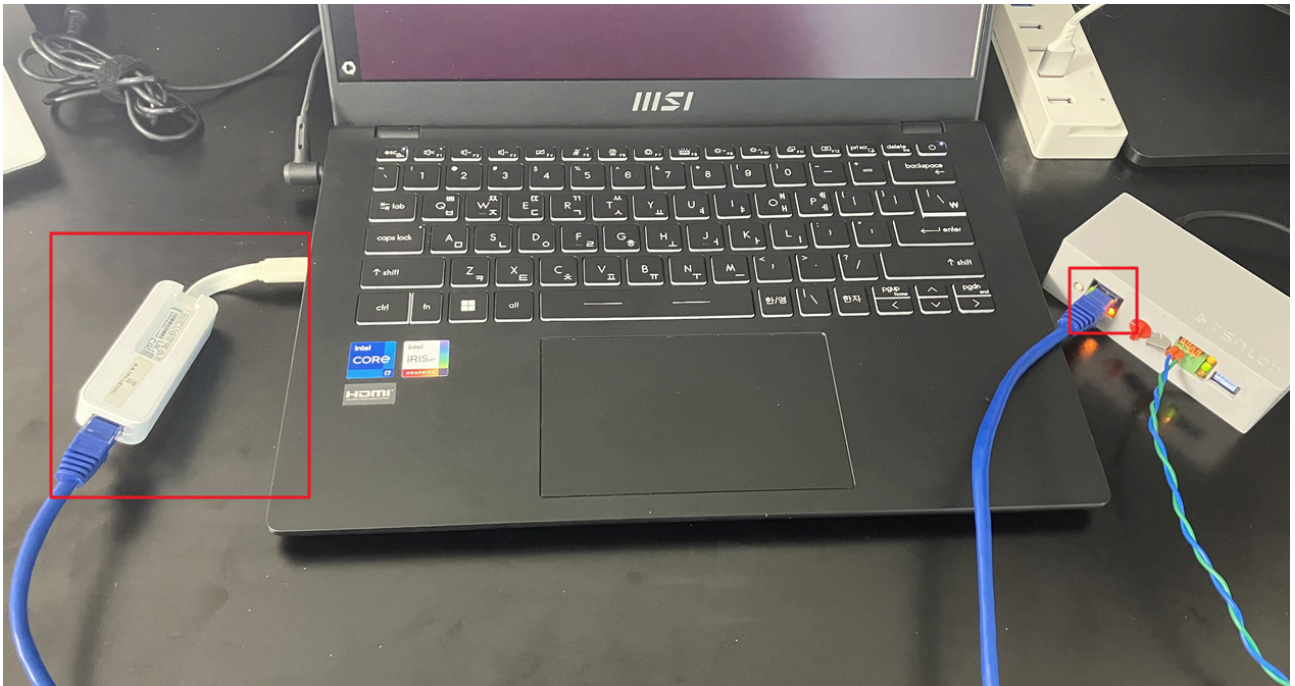
<pre>tsnlab@tsnlab-t1s-node-1:~ \$ ping 10.1.1.2 PING 10.1.1.2 (10.1.1.2) 56(84) bytes of data. 64 bytes from 10.1.1.2: icmp_seq=1 ttl=64 time=0.686 ms 64 bytes from 10.1.1.2: icmp_seq=2 ttl=64 time=0.628 ms 64 bytes from 10.1.1.2: icmp_seq=3 ttl=64 time=0.631 ms 64 bytes from 10.1.1.2: icmp_seq=4 ttl=64 time=0.639 ms 64 bytes from 10.1.1.2: icmp_seq=5 ttl=64 time=0.643 ms 64 bytes from 10.1.1.2: icmp_seq=6 ttl=64 time=0.661 ms 64 bytes from 10.1.1.2: icmp_seq=7 ttl=64</pre>	<pre>tsnlab@tsnlab-t1s-node-2:~ \$ sudo tcpdump -i eth1 -ne tcpdump: verbose output suppressed, use -v[v]... for full protocol decode listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes 03:08:26.566726 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 98: 10.1.1.1 > 10.1.1.2: ICMP echo request, id 3, seq 1, length 64 03:08:26.566785 d0:d1:95:30:23:02 > d0:d1:95:30:23:01, ethertype IPv4 (0x0800), length 98: 10.1.1.2 > 10.1.1.1: ICMP echo reply, id 3, seq 1, length 64 03:08:27.578426 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 98: 10.1.1.1 > 10.1.1.2: ICMP</pre>
---	---

<10Base-T1S 네트워크 중 HAT 보드 간에 ICMP Ping-Pong이 이뤄지는 모습

(왼쪽) 중간에 위치한 HAT 보드의 RPi 터미널 / (오른쪽) 가장 오른쪽에 위치한 HAT 보드의 RPi 터미널>

네트워크의 중간에 위치한 HAT 보드와 가장 오른쪽 끝에 위치한 HAT 보드 간에 ping 명령으로 서로 10Base-T1S를 통한 통신이 가능한 것을 확인할 수 있습니다. 다시 말해, Converter 보드의 코디네이터에서 발생하는 신호를 기반으로 팔로워인 HAT 보드간에는 10Base-T1S를 통한 이더넷 통신(ping과 tcpdump 명령 활용)이 가능한 것을 알 수 있습니다.

이제, 일반 PC를 Converter 보드와 연결해 10Base-T1S 네트워크에 참여시킵니다.



<일반 PC와 Converter 보드가 LAN Cable을 통해 연결된 모습>

Converter 보드의 RJ45 규격의 LAN 인터페이스를 활용해 일반 PC를 연결하게 되면, 일반 PC는 10Base-T1S 네트워크에 참여할 수 있게 됩니다. 다시 말해, 일반 PC는 10Base-T1S 네트워크 상에 존재하는 특정 노드와 이더넷 통신을 수행할 수 있게 됩니다.

```
pakji@TSNLabMachine:~$ ifconfig enx8e43bad21a9
enx8e43bad21a9: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.1.1.3 netmask 255.255.255.0 broadcast 10.1.1.255
    inet6 fe80::db47:4241:2f3:f1a4 prefixlen 64 scopeid 0x20<link>
    ether f8:e4:3b:ad:21:a9 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

<일반 PC의 네트워크 인터페이스에 적용된 IPv4 주소>

이더넷 통신을 위해서 일반 PC의 LAN 포트에 해당하는 네트워크 인터페이스에 `ifconfig` 혹은 `nmtui` 명령으로 10Base-T1S 네트워크의 IPv4 대역과 동일한 대역의 주소를 할당시킵니다.

※ 본 예제에서는 10Base-T1S 네트워크의 IPv4 대역을 10.1.1.X로 하였으며, 일반 PC에는 10.1.1.3 중간 HAT 보드에는 10.1.1.1 그리고 가장 오른쪽의 HAT 보드에는 10.1.1.2를 할당시켰습니다.

```

pakji@TSNLabMachine:~$ ping 10.1.1.1
PING 10.1.1.1 (10.1.1.1) 56(84) bytes of data.
64 bytes from 10.1.1.1: icmp_seq=1 ttl=64 time=1.86 ms
64 bytes from 10.1.1.1: icmp_seq=2 ttl=64 time=1.92 ms
64 bytes from 10.1.1.1: icmp_seq=3 ttl=64 time=2.02 ms
^C
--- 10.1.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 1.864/1.935/2.017/0.062 ms
pakji@TSNLabMachine:~$ ping 10.1.1.2
PING 10.1.1.2 (10.1.1.2) 56(84) bytes of data.
64 bytes from 10.1.1.2: icmp_seq=1 ttl=64 time=1.91 ms
64 bytes from 10.1.1.2: icmp_seq=2 ttl=64 time=1.83 ms
64 bytes from 10.1.1.2: icmp_seq=3 ttl=64 time=1.83 ms
^C
--- 10.1.1.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.827/1.855/1.911/0.039 ms

```

<일반 PC와 10Base-T1S 네트워크 상에 존재하는 노드 간에 ICMP Ping-Pong을 수행한 모습>

LAN 연결 후 IPv4 주소를 할당하였다면, 일반 PC가 Converter 보드를 통해 10Base-T1S 네트워크 상에 존재하는 노드와 ICMP Ping-Pong이 수행할 수 있는 것을 ping 명령을 통해 확인 가능합니다.

```

TSNLabMachine:~$ iperf3 -c 10.1.1.1 -u -b 9.4Mb
Connecting to host 10.1.1.1, port 5201
local 10.1.1.3 port 41472 connected to 10.1.1.1 port 5201
Interval      Transfer      Bitrate      Total Datagrams
0.00-1.00    sec  1.12 MBytes  9.39 Mbits/sec  811
1.00-2.00    sec  1.12 MBytes  9.41 Mbits/sec  812
2.00-3.00    sec  1.12 MBytes  9.39 Mbits/sec  811
3.00-4.00    sec  1.12 MBytes  9.41 Mbits/sec  812
4.00-5.00    sec  1.12 MBytes  9.39 Mbits/sec  811
5.00-6.00    sec  1.12 MBytes  9.41 Mbits/sec  812
6.00-7.00    sec  1.12 MBytes  9.39 Mbits/sec  811
7.00-8.00    sec  1.12 MBytes  9.39 Mbits/sec  811
8.00-9.00    sec  1.12 MBytes  9.41 Mbits/sec  812
9.00-10.00   sec  1.12 MBytes  9.39 Mbits/sec  811
Interval      Transfer      Bitrate      Jitter      Lost/Tot
0.00-10.00   sec  11.2 MBytes  9.40 Mbits/sec  0.000 ms  0/8114
0.00-10.00   sec  11.2 MBytes  9.40 Mbits/sec  0.316 ms  0/8114

tsnlab@tsnlab-t1s-node-1:~$ iperf3 -s
Server listening on 5201 (test #1)
Accepted connection from 10.1.1.3, port 60058
[ 5] local 10.1.1.1 port 5201 connected to 10.1.1.3 port 41472
[ ID] Interval      Transfer      Bitrate      Jitter
[ 5] 0.00-1.00    sec  1.12 MBytes  9.36 Mbits/sec  0.321 ms
[ 5] 1.00-2.00    sec  1.12 MBytes  9.39 Mbits/sec  0.339 ms
[ 5] 2.00-3.00    sec  1.12 MBytes  9.41 Mbits/sec  0.322 ms
[ 5] 3.00-4.00    sec  1.12 MBytes  9.39 Mbits/sec  0.341 ms
[ 5] 4.00-5.00    sec  1.12 MBytes  9.41 Mbits/sec  0.323 ms
[ 5] 5.00-6.00    sec  1.12 MBytes  9.39 Mbits/sec  0.329 ms
[ 5] 6.00-7.00    sec  1.12 MBytes  9.41 Mbits/sec  0.317 ms
[ 5] 7.00-8.00    sec  1.12 MBytes  9.39 Mbits/sec  0.333 ms
[ 5] 8.00-9.00    sec  1.12 MBytes  9.39 Mbits/sec  0.315 ms
[ 5] 9.00-10.00   sec  1.12 MBytes  9.41 Mbits/sec  0.330 ms
[ 5] 10.00-10.00  sec  2.83 KBytes  12.4 Mbits/sec  0.316 ms
[ ID] Interval      Transfer      Bitrate      Jitter
[ 5] 0.00-10.00   sec  11.2 MBytes  9.40 Mbits/sec  0.316 ms

```

<일반 PC에서 10Base-T1S 네트워크 상에 존재하는 노드로 UDP 트래픽을 전송하는 모습>

LAN 연결 후 IPv4 주소를 할당하였다면, 일반 PC가 Converter 보드를 통해 10Base-T1S 네트워크 상에 존재하는 노드로 UDP 트래픽을 10Mbps 정도의 속도로 전송할 수 있는 것을 iperf3 명령을 통해 확인 가능합니다.


```

TSNLabMachine:~$ iperf3 -s
-----
listening on 5201 (test #1)
-----
Accepted connection from 10.1.1.1, port 53904
local 10.1.1.3 port 5201 connected to 10.1.1.1 port 44050
Interval      Transfer      Bitrate      Jitter      Lost/Total Dat
0.00-1.00    sec  1.12 MBytes  9.41 Mbits/sec  0.140 ms  22/834 (2.6%)
1.00-2.00    sec  1.13 MBytes  9.44 Mbits/sec  0.070 ms  64/879 (7.3%)
2.00-3.00    sec  1.13 MBytes  9.44 Mbits/sec  0.143 ms  50/865 (5.8%)
3.00-4.00    sec  1.13 MBytes  9.44 Mbits/sec  0.132 ms  47/862 (5.5%)
4.00-5.00    sec  1.13 MBytes  9.44 Mbits/sec  0.157 ms  48/863 (5.6%)
5.00-6.00    sec  1.13 MBytes  9.44 Mbits/sec  0.122 ms  48/863 (5.6%)
6.00-7.00    sec  1.13 MBytes  9.44 Mbits/sec  0.118 ms  48/863 (5.6%)
7.00-8.00    sec  1.13 MBytes  9.44 Mbits/sec  0.152 ms  49/864 (5.7%)
8.00-9.00    sec  1.13 MBytes  9.44 Mbits/sec  0.127 ms  48/863 (5.6%)
9.00-10.00   sec  1.13 MBytes  9.44 Mbits/sec  0.150 ms  48/863 (5.6%)

tsnlab@tsnlab-t1s-node-1:~$ iperf3 -c 10.1.1.3 -u -b 10M
Connecting to host 10.1.1.3, port 5201
[ 5] local 10.1.1.1 port 44050 connected to 10.1.1.3 port 5201
[ ID] Interval      Transfer      Bitrate      Total Data
[ 5] 0.00-1.00    sec  1.19 MBytes  10.0 Mbits/sec  865
[ 5] 1.00-2.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 2.00-3.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 3.00-4.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 4.00-5.00    sec  1.19 MBytes  10.0 Mbits/sec  864
[ 5] 5.00-6.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 6.00-7.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 7.00-8.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 8.00-9.00    sec  1.19 MBytes  10.0 Mbits/sec  864
[ 5] 9.00-10.00   sec  1.19 MBytes  10.0 Mbits/sec  863
[ ID] Interval      Transfer      Bitrate      Jitter
[ 5] 0.00-10.00   sec  11.9 MBytes  10.0 Mbits/sec  0.00
[ 5] 0.00-10.01   sec  11.3 MBytes  9.44 Mbits/sec  0.17

```

<일반 PC에서 10Base-T1S 네트워크 상에 존재하는 노드로부터 UDP 트래픽을 수신하는 모습>

LAN 연결 후 IPv4 주소를 할당하였다면, 일반 PC가 Converter 보드를 통해 10Base-T1S 네트워크 상에 존재하는 노드로부터 UDP 트래픽을 10Mbps 정도의 속도로 수신할 수 있는 것을 iperf3 명령을 통해 확인 가능합니다.

※ 이와 같은 Converter 보드 기능을 활용해 일반 PC를 10Base-T1S의 게이트웨이로 동작시키거나 혹은 10Base-T1S 네트워크의 프로토콜 실험 용도로 활용할 수 있습니다.

```

tsnlab@tsnlab-t1s-node-1:~$ iperf3 -c 10.1.1.2 -u -b 10Mb
Connecting to host 10.1.1.2, port 5201
[ 5] local 10.1.1.1 port 53685 connected to 10.1.1.2 port 5201
[ ID] Interval      Transfer      Bitrate      Total Data
[ 5] 0.00-1.00    sec  1.19 MBytes  10.0 Mbits/sec  865
[ 5] 1.00-2.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 2.00-3.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 3.00-4.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 4.00-5.00    sec  1.19 MBytes  10.0 Mbits/sec  864
[ 5] 5.00-6.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 6.00-7.00    sec  1.19 MBytes  10.0 Mbits/sec  863
[ 5] 7.00-8.00    sec  1.19 MBytes  9.99 Mbits/sec  863
[ 5] 8.00-9.00    sec  1.19 MBytes  10.0 Mbits/sec  864
[ 5] 9.00-10.00   sec  1.19 MBytes  9.99 Mbits/sec  863
[ ID] Interval      Transfer      Bitrate      Jitter
[ 5] 0.00-10.00   sec  11.9 MBytes  10.0 Mbits/sec  0.000 ms
[ 5] 0.00-10.01   sec  11.3 MBytes  9.44 Mbits/sec  0.122 ms

tsnlab@tsnlab-t1s-node-2:~$ iperf3 -s
-----
Server listening on 5201 (test #1)
-----
Accepted connection from 10.1.1.1, port 52078
[ 5] local 10.1.1.2 port 5201 connected to 10.1.1.1
[ ID] Interval      Transfer      Bitrate
[ 5] 0.00-1.00    sec  1.12 MBytes  9.42 Mbits/sec
[ 5] 1.00-2.00    sec  1.13 MBytes  9.44 Mbits/sec
[ 5] 2.00-3.00    sec  1.13 MBytes  9.44 Mbits/sec
[ 5] 3.00-4.00    sec  1.13 MBytes  9.44 Mbits/sec
[ 5] 4.00-5.00    sec  1.12 MBytes  9.43 Mbits/sec
[ 5] 5.00-6.00    sec  1.13 MBytes  9.44 Mbits/sec
[ 5] 6.00-7.00    sec  1.13 MBytes  9.44 Mbits/sec
[ 5] 7.00-8.00    sec  1.13 MBytes  9.44 Mbits/sec
[ 5] 8.00-9.00    sec  1.13 MBytes  9.44 Mbits/sec
[ 5] 9.00-10.00   sec  1.13 MBytes  9.44 Mbits/sec
[ 5] 10.00-10.01   sec  7.07 KBytes  9.99 Mbits/sec

```

<일반 PC를 제외한 10Base-T1S 노드간의 통신을 수행하는 모습>

```

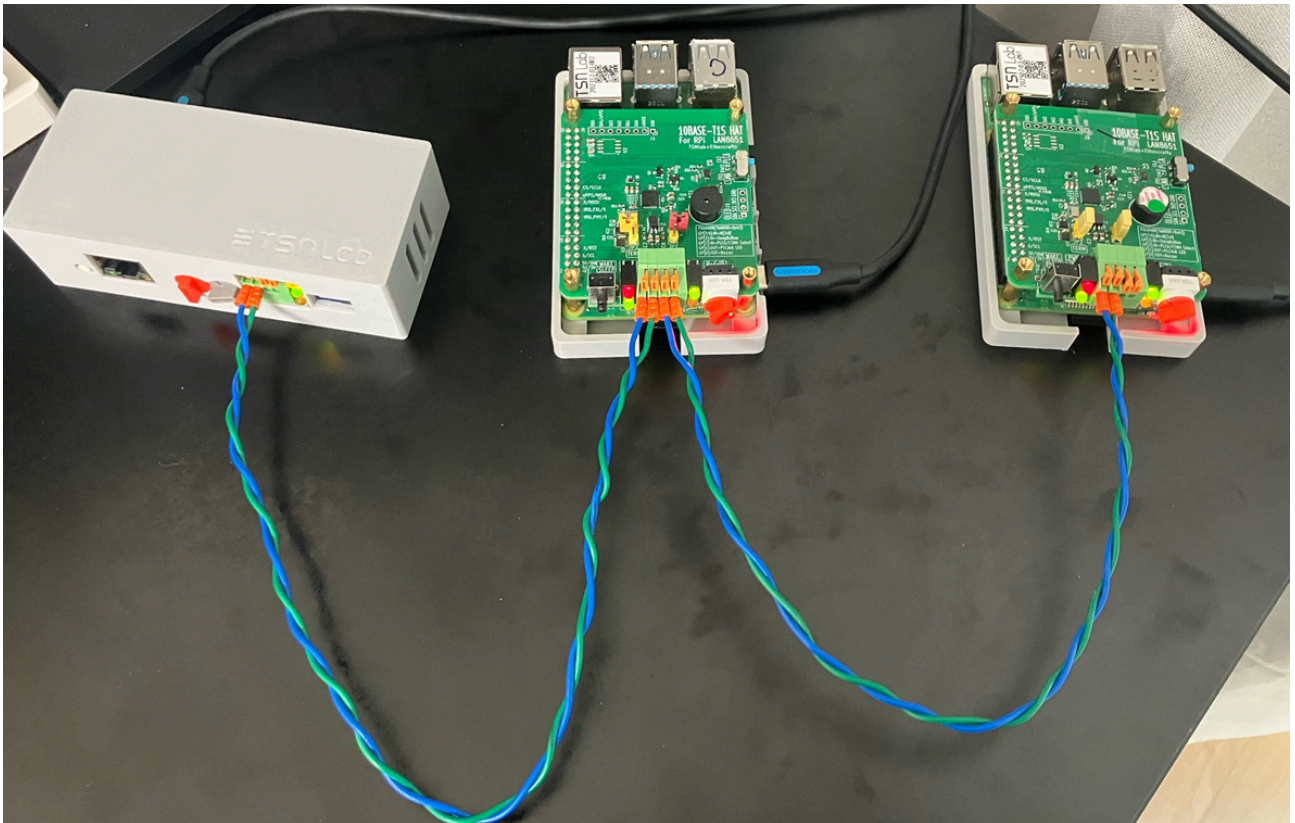
pakji@TSNLabMachine:~$ sudo tcpdump -i eth1 -ne
tcpdump: eth1: No such device exists
(No such device exists)
pakji@TSNLabMachine:~$ ^Cconfig enx8e43bad21a9
pakji@TSNLabMachine:~$ ^C
pakji@TSNLabMachine:~$ sudo tcpdump -i enx8e43bad21a9 -ne
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enx8e43bad21a9, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10:57:32.615593 f8:e4:3b:ad:21:a9 > 33:33:00:00:00:02, ethertype IPv6 (0x86dd), length 62:
f3:f1a4 > ff02::2: ICMP6, router solicitation, length 8

```

<일반 PC에서 tcpdump를 통해 10Base-T1S 네트워크에 존재하는 패킷을 캡처하는 모습>

10Base-T1S 네트워크 상에 이더넷 프레임(IP, TCP, UDP 등)이 존재할 경우 Converter 보드는 이를 스니핑하여 일반 PC로 스니핑된 프레임을 전송할 수 있습니다. 이러한 기능은 코디네이터와 팔로워 모드로는 실행이 불가하며 본 상품에서 제공하는 특별한 스니퍼(Sniffer) 모드를 통해서 활성화시킬 수 있습니다. 이에 대한 설명은 다음 4.2.(일반 PC를 활용한 10Base-T1S 네트워크 패킷 스니핑)에서 진행합니다.

4.2. 일반 PC를 활용한 10Base-T1S 네트워크 패킷 스니핑

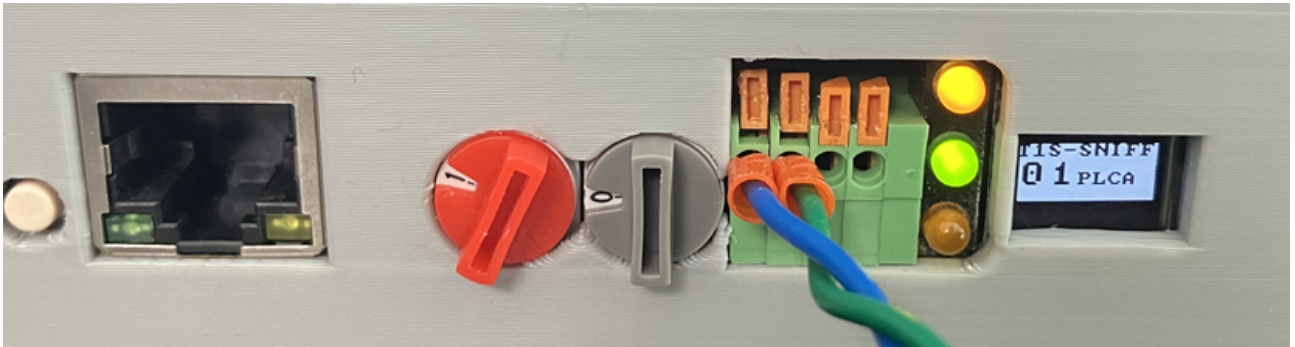


<버스 구조의 10Base-T1S 네트워크 패킷 스니핑 예제>

본 4.2. 과정에서는 일반 PC가 Converter 보드를 활용하여 10Base-T1S 네트워크 상에 존재하는 패킷(=이더넷 프레임)들을 스니핑하는 예제를 설명합니다. 이를 위한 10Base-T1S 네트워크를 형성하기 위해서 Converter 보드(= 10Base-T1S Converter) 한 개와 HAT 보드(= 10Base-T1S HAT) 두 개를 사용합니다.

※ 일반 PC로는 MSI의 Modern 14 모델의 노트북을 사용하였으며, 노트북의 운영체제로는 Linux/Ubuntu 24.04 LTS를 활용하였습니다.

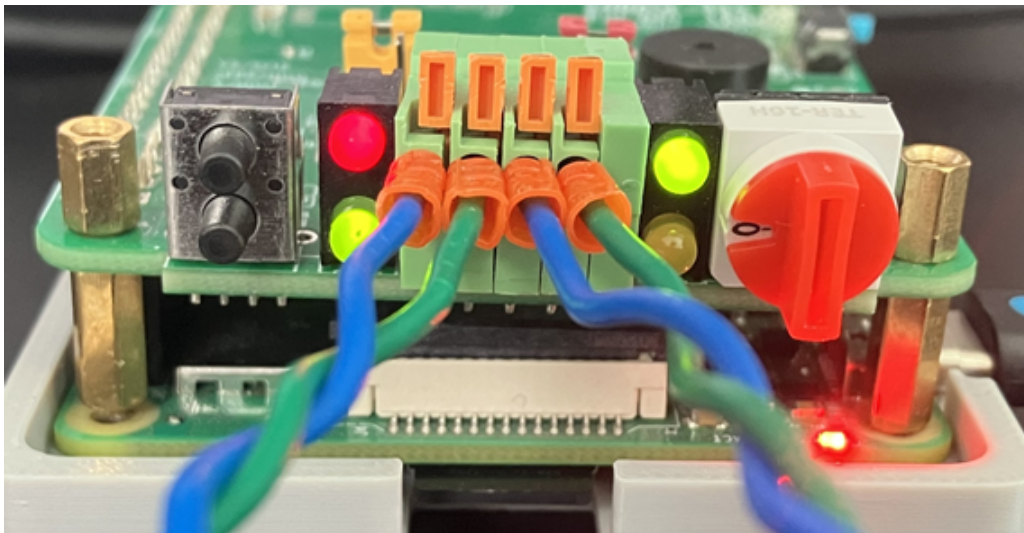
※ 유의사항: 10Base-T1S 네트워크 환경 구성은 사용자 환경에 맞춰 구성해주시면 됩니다. 만약, 10Base-T1S HAT 보드를 사용하고 계신다면 [HAT 보드 사용자 매뉴얼](#)을 참고해주시기 바랍니다.



<10Base-T1S 네트워크 중 가장 왼쪽에 위치한 Converter 보드의 인터페이스>

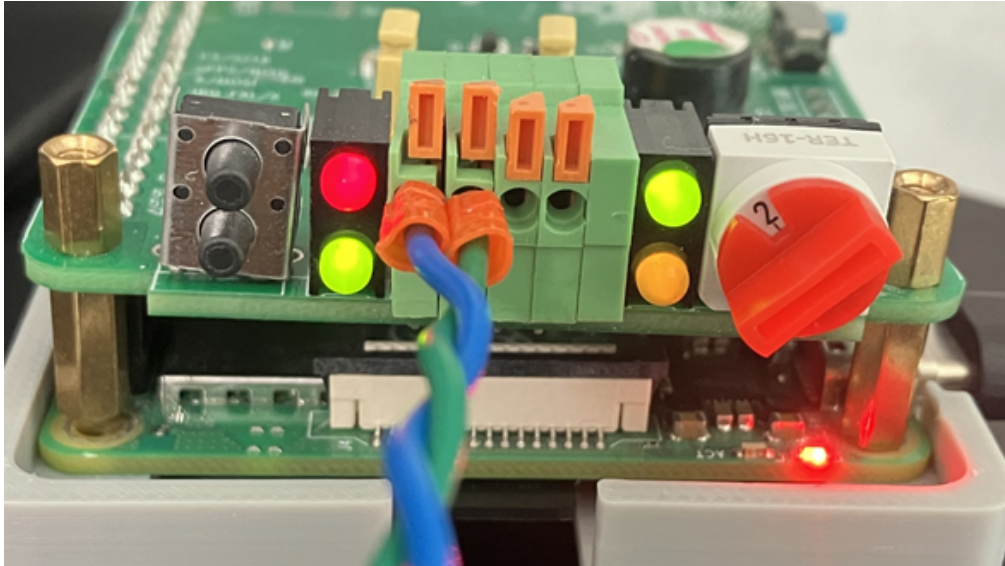
Converter 보드의 Node ID는 1로 설정하였으며, Node Count는 0으로 설정(팔로워 모드 이면서, Node Count가 0일 경우 스니퍼 모드로 동작)하였습니다. 이 값이 의미하는 것은 “Converter 보드가 10Base-T1S 네트워크의 팔로워 역할을 수행하면서 통신에는 참여하지는 않고, 패킷 스니핑만 수행한다.”와 같습니다. 또한, Converter 보드의 RJ-45 규격의 LAN 인터페이스는 아직 일반 PC와 연결하지 않은 상태입니다.

※ Converter 보드가 스니퍼 모드일 경우, OLED Display의 상단에 TIS-SNIFF라고 출력됩니다.



<10Base-T1S 네트워크 중 중간에 위치한 HAT 보드의 인터페이스>

중간에 위치한 HAT 보드의 Node ID는 0으로 설정하였습니다(HAT의 경우 Node Count는 8로 고정). 이 값이 의미하는 것은 “HAT 보드가 10Base-T1S 네트워크의 코디네이터 역할을 수행하여 팔로워를 코디네이터의 신호에 맞춰 통신을 수행시킨다.”와 같습니다.



<10Base-T1S 네트워크 중 가장 오른쪽에 위치한 HAT 보드의 인터페이스>

오른쪽 끝에 위치한 HAT 보드의 Node ID는 2로 설정하였습니다(HAT의 경우 Node Count는 8로 고정). 이 값이 의미하는 것은 “HAT 보드가 10Base-T1S 네트워크의 팔로워 역할을 수행하며 코디네이터의 신호에 맞춰 통신을 수행한다.”와 같습니다.

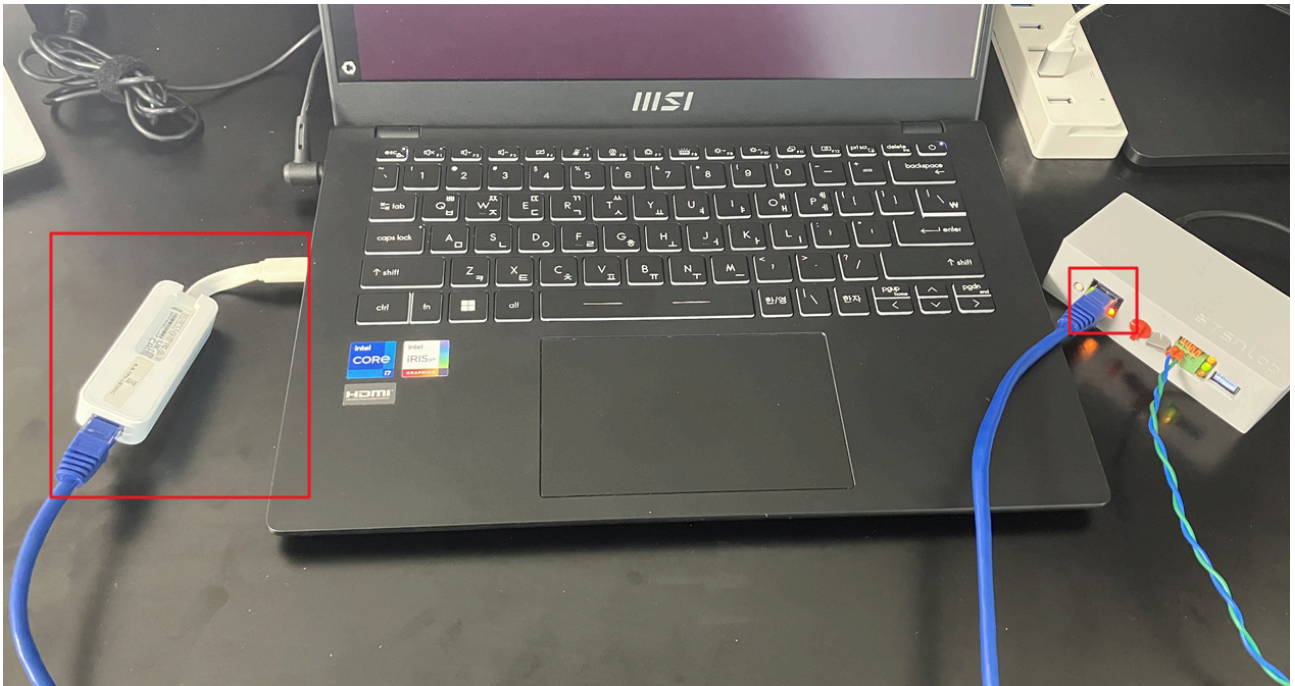
<pre>tsnlab@tsnlab-t1s-node-1:~ \$ ping 10.1.1.2 PING 10.1.1.2 (10.1.1.2) 56(84) bytes of data. 64 bytes from 10.1.1.2: icmp_seq=1 ttl=64 time=0.686 ms 64 bytes from 10.1.1.2: icmp_seq=2 ttl=64 time=0.628 ms 64 bytes from 10.1.1.2: icmp_seq=3 ttl=64 time=0.631 ms 64 bytes from 10.1.1.2: icmp_seq=4 ttl=64 time=0.639 ms 64 bytes from 10.1.1.2: icmp_seq=5 ttl=64 time=0.643 ms 64 bytes from 10.1.1.2: icmp_seq=6 ttl=64 time=0.661 ms 64 bytes from 10.1.1.2: icmp_seq=7 ttl=64</pre>	<pre>tsnlab@tsnlab-t1s-node-2:~ \$ sudo tcpdump -i eth1 -ne tcpdump: verbose output suppressed, use -v[v]... for full protocol decode listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes 03:08:26.566726 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 98: 10.1.1.1 > 10.1.1.2: ICMP echo request, id 3, seq 1, length 64 03:08:26.566785 d0:d1:95:30:23:02 > d0:d1:95:30:23:01, ethertype IPv4 (0x0800), length 98: 10.1.1.2 > 10.1.1.1: ICMP echo reply, id 3, seq 1, length 64 03:08:27.578426 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 98: 10.1.1.1 > 10.1.1.2: ICMP</pre>
---	---

<10Base-T1S 네트워크 중 HAT 보드 간에 ICMP Ping-Pong이 이뤄지는 모습

(왼쪽) 중간에 위치한 HAT 보드의 RPi 터미널 / (오른쪽) 가장 오른쪽에 위치한 HAT 보드의 RPi 터미널>

네트워크의 중간에 위치한 HAT 보드와 가장 오른쪽 끝에 위치한 HAT 보드 간에 ping 명령으로 서로 10Base-T1S를 통한 통신이 가능한 것을 확인할 수 있습니다. 다시 말해, 중간 HAT 보드의 코디네이터에서 발생하는 신호를 기반으로 HAT 보드간에는 10Base-T1S를 통한 이더넷 통신(ping과 tcpdump 명령 활용)이 가능한 것을 알 수 있습니다.

이제, 일반 PC를 Converter 보드와 연결해 10Base-T1S 네트워크에 참여시킵니다.



<일반 PC와 Converter 보드가 LAN Cable을 통해 연결된 모습>

Converter 보드의 RJ45 규격의 LAN 인터페이스를 활용해 일반 PC를 연결하게 되면, 일반 PC는 10Base-T1S 네트워크에 참여할 수 있게 됩니다.


```

pakji@TSNLabMachine:~$ ping 10.1.1.1
PING 10.1.1.1 (10.1.1.1) 56(84) bytes of data.
From 10.1.1.3 icmp_seq=1 Destination Host Unreachable
From 10.1.1.3 icmp_seq=2 Destination Host Unreachable
From 10.1.1.3 icmp_seq=3 Destination Host Unreachable
^C
--- 10.1.1.1 ping statistics ---
4 packets transmitted, 0 received, +3 errors, 100% packet loss,
pipe 4
pakji@TSNLabMachine:~$ ping 10.1.1.2
PING 10.1.1.2 (10.1.1.2) 56(84) bytes of data.
From 10.1.1.3 icmp_seq=1 Destination Host Unreachable
From 10.1.1.3 icmp_seq=2 Destination Host Unreachable
From 10.1.1.3 icmp_seq=3 Destination Host Unreachable
From 10.1.1.3 icmp_seq=4 Destination Host Unreachable
^C
--- 10.1.1.2 ping statistics ---
4 packets transmitted, 0 received, +4 errors, 100% packet loss,
pipe 4

```

<일반 PC에서 10Base-T1S 노드들과 ICMP Ping-Pong이 수행되지 않는 모습>

※ 본 예제에서는 10Base-T1S 네트워크의 IPv4 대역을 10.1.1.X로 하였으며, 일반 PC에는 10.1.1.3 중간 HAT 보드에는 10.1.1.1 그리고 가장 오른쪽의 HAT 보드에는 10.1.1.2를 할당시켰습니다.

Converter 보드의 모드를 스니퍼 모드로 설정하였기 때문에 일반 PC는 10Base-T1S 네트워크에 참석해서 통신을 수행할 수 없습니다.

<pre> tsnlab@tsnlab-t1s-node-1:~ \$ ping 10.1.1.2 PING 10.1.1.2 (10.1.1.2) 56(84) bytes of data. 64 bytes from 10.1.1.2: icmp_seq=1 ttl=64 time =0.685 ms 64 bytes from 10.1.1.2: icmp_seq=2 ttl=64 time =0.659 ms 64 bytes from 10.1.1.2: icmp_seq=3 ttl=64 time =0.586 ms 64 bytes from 10.1.1.2: icmp_seq=4 ttl=64 time =0.607 ms 64 bytes from 10.1.1.2: icmp_seq=5 ttl=64 time =0.561 ms 64 bytes from 10.1.1.2: icmp_seq=6 ttl=64 time =0.556 ms </pre>	<pre> tsnlab@tsnlab-t1s-node-2:~ \$ ping 10.1.1.1 PING 10.1.1.1 (10.1.1.1) 56(84) bytes of data. 64 bytes from 10.1.1.1: icmp_seq=1 ttl=64 ti me=0.657 ms 64 bytes from 10.1.1.1: icmp_seq=2 ttl=64 ti me=0.601 ms 64 bytes from 10.1.1.1: icmp_seq=3 ttl=64 ti me=0.558 ms 64 bytes from 10.1.1.1: icmp_seq=4 ttl=64 ti me=0.593 ms 64 bytes from 10.1.1.1: icmp_seq=5 ttl=64 ti me=0.624 ms 64 bytes from 10.1.1.1: icmp_seq=6 ttl=64 ti </pre>
---	---

<10Base-T1S 네트워크에 존재하는 HAT 보드 간에 ICMP Ping-Pong을 수행하는 모습>

```

tsnlab@tsnlab-t1s-node-1:~$ iperf3 -c 10.1.1.2 -u -b 10M
Connecting to host 10.1.1.2, port 5201
[ 5] local 10.1.1.1 port 45995 connected to 10.1.1.2 port 5201
[ ID] Interval           Transfer     Bitrate     Total
[ 5] 0.00-1.00 sec      1.19 MBytes 10.0 Mbits/sec 865
[ 5] 1.00-2.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 2.00-3.00 sec      1.19 MBytes 9.99 Mbits/sec 863
[ 5] 3.00-4.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 4.00-5.00 sec      1.19 MBytes 10.0 Mbits/sec 864
[ 5] 5.00-6.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 6.00-7.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 7.00-8.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 8.00-9.00 sec      1.19 MBytes 10.0 Mbits/sec 864
[ 5] 9.00-10.00 sec     1.19 MBytes 9.99 Mbits/sec 863
[ ID] Interval           Transfer     Bitrate     Jitter
[ 5] 0.00-10.00 sec     11.9 MBytes 10.0 Mbits/sec 0.0
0/8634 (0%) sender
[ 5] 0.00-10.01 sec     11.2 MBytes 9.42 Mbits/sec 0.1
489/8628 (5.7%) receiver

tsnlab@tsnlab-t1s-node-2:~$ iperf3 -s
-----
Server listening on 5201 (test #1)
-----
Accepted connection from 10.1.1.1, port 37826
[ 5] local 10.1.1.2 port 5201 connected to 10.1.1.1 port 37826
[ ID] Interval           Transfer     Bitrate     Jitter
[ 5] 0.00-1.00 sec      1.12 MBytes 9.40 Mbits/sec 0.121 ms
22/834 (2.6%)
[ 5] 1.00-2.00 sec      1.12 MBytes 9.43 Mbits/sec 0.106 ms
70/884 (7.9%)
[ 5] 2.00-3.00 sec      1.12 MBytes 9.43 Mbits/sec 0.178 ms
47/861 (5.5%)
[ 5] 3.00-4.00 sec      1.12 MBytes 9.42 Mbits/sec 0.134 ms
48/861 (5.6%)
[ 5] 4.00-5.00 sec      1.12 MBytes 9.43 Mbits/sec 0.152 ms
50/864 (5.8%)
[ 5] 5.00-6.00 sec      1.12 MBytes 9.42 Mbits/sec 0.

```

<10Base-T1S 네트워크에 존재하는 HAT 보드 간에 UDP 트래픽을 송수신하는 모습>

```

pakji@TSNLabMachine:~$ sudo tcpdump -i enx8e43bad21a9 -ne
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enx8e43bad21a9, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:33:44.066975 d0:d1:95:30:23:02 > d0:d1:95:30:23:01, ethertype ARP (0x0806), length 60: Request who-has 10.1.1.1 tell 10.1.1.2, length 46
13:33:44.066978 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype ARP (0x0806), length 60: Reply 10.1.1.1 is-at d0:d1:95:30:23:01, length 46
13:33:44.093871 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 98: 10.1.1.1 > 10.1.1.2: ICMP echo request, id 5, seq 6, length 64
13:33:44.093874 d0:d1:95:30:23:02 > d0:d1:95:30:23:01, ethertype IPv4 (0x0800), length 98: 10.1.1.2 > 10.1.1.1: ICMP echo reply, id 5, seq 6, length 64
13:33:44.221609 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype ARP (0x0806), length 60: Request who-has 10.1.1.2 tell 10.1.1.1, length 46
13:33:44.221612 d0:d1:95:30:23:02 > d0:d1:95:30:23:01, ethertype ARP (0x0806), length 60: Reply 10.1.1.2 is-at d0:d1:95:30:23:02, length 46
13:33:44.643133 d0:d1:95:30:23:02 > d0:d1:95:30:23:01, ethertype IPv4 (0x0800), length 98: 10.1.1.2 > 10.1.1.1: ICMP echo request, id 1, seq 6, length 64
13:33:44.643136 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 98: 10.1.1.1 > 10.1.1.2: ICMP echo reply, id 1, seq 6, length 64
13:33:45.117854 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 98: 10.1.1.1 > 10.1.1.2: ICMP echo request, id 5, seq 7, length 64
13:33:45.117857 d0:d1:95:30:23:02 > d0:d1:95:30:23:01, ethertype IPv4 (0x0800), length 98: 10.1.1.2 > 10.1.1.1: ICMP echo reply, id 5, seq 7, length 64

```

<일반 PC에서 10Base-T1S 네트워크 상의 ICMP Ping-Pong 패킷을 스니핑하는 모습>

```

pakji@TSNLabMachine:~$ sudo tcpdump -i enx8e43bad21a9 -ne
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enx8e43bad21a9, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:35:49.635118 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 1490: 10.1.1.1.45995 > 10.1.1.2.5201: UDP, length 1448
13:35:49.636293 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 1490: 10.1.1.1.45995 > 10.1.1.2.5201: UDP, length 1448
13:35:49.637470 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 1490: 10.1.1.1.45995 > 10.1.1.2.5201: UDP, length 1448
13:35:49.638669 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 1490: 10.1.1.1.45995 > 10.1.1.2.5201: UDP, length 1448
13:35:49.640176 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 1490: 10.1.1.1.45995 > 10.1.1.2.5201: UDP, length 1448
13:35:49.641432 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 1490: 10.1.1.1.45995 > 10.1.1.2.5201: UDP, length 1448
13:35:49.642593 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 1490: 10.1.1.1.45995 > 10.1.1.2.5201: UDP, length 1448
13:35:49.643681 d0:d1:95:30:23:01 > d0:d1:95:30:23:02, ethertype IPv4 (0x0800), length 1490: 10.1.1.1.45995 > 10.1.1.2.5201: UDP, length 1448

```

<일반 PC에서 10Base-T1S 네트워크 상의 UDP 트래픽을 스니핑하는 모습>

Converter 보드의 스니퍼 모드에서는 10Base-T1S 노드간에 주고받는 패킷을 스니핑하는 것을 지원하므로 일반 PC에서 패킷 캡처 툴(e.g. tcpdump, wireshark 등)을 활용해 10Base-T1S 네트워크에 흐르는 패킷을 캡처하여 살펴볼 수 있습니다.

※ 이와 같은 Converter 보드 기능을 활용해 일반 PC를 10Base-T1S 네트워크 상의 이상 데이터 탐지 혹은 프로토콜 검증 등을 위해 활용할 수 있습니다.

4. 이슈 리스트

4.1. ETH 포트 전원 이슈

간헐적으로 Converter 보드의 **ETH** 포트가 가끔씩 살아나지 않는 이슈가 있습니다. 이때에는 **Reset** 버튼을 눌러주면 LAN 포트가 다시 살아나게 됩니다.

4.2. 스니퍼 & 코디네이터 모드 충돌 이슈

Converter 보드가 코디네이터(Node ID = 0)로 동작할 때는 스니퍼 모드(Node Count = 0)로 동작할 수 없습니다. 10Base-T1S 버스 중 코디네이터는 Node Count 개수에 따라 PLCA의 주기를 설정하기 때문에 스니퍼 모드의 Node Count = 0과 충돌됩니다. 따라서, **스니퍼 모드는 코디네이터 모드에서 동작 불가**하며 반드시 스니퍼 모드는 팔로워 모드에서만 동작 가능합니다.

4.3. 양방향 통신 대역폭 이슈

2025-12-29 기준, Converter 보드를 활용해 **10Base-T1S** 노드와 일반 **PC** 간에 양방향 통신을 수행하는 경우 **Tx/Rx**가 정상동작 하지 않는 이슈가 존재합니다. 이는 LAN8670 칩에서 발생하는 이슈로 소프트웨어로 해결이 불가능한 상태입니다.